

01. LangChain : AI 연결의 패러다임 시프트

거대언어모델(LLM)과 다양한 외부 도구를 연결하여 실질적인 자율형 비즈니스 서비스를 구축하는 오픈소스 엔진

ETYMOLOGY & DEFINITION

Language Model + Chain

언어 모델에 강력한 "연결 고리"를 더하다

LLM은 홀로 존재할 때 똑똑한 두뇌에 불과하지만, 주변 환경과 연결되지 못해 고립되어 있습니다. 랭체인은 이 똑똑한 두뇌에 손과 발(API), 기억(Memory), 그리고 눈(Data)을 달아주는 파이프라인 프레임워크입니다.

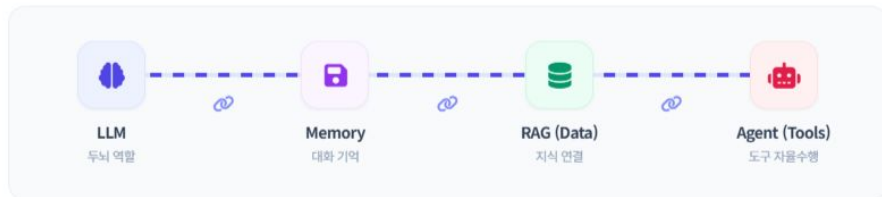
CORE VALUE

각기 규격이 다른 AI 모델, 데이터베이스, 웹 API 등을 유기적인 체인처럼 조합하여 실전 비즈니스 가치를 자동 산출합니다.

ARCHITECTURE VISUALIZATION

랭체인이 설계하는 무결한 연결 파이프라인

외부 인프라를 체인으로 엮어 완성하는 현대적 AI 오케스트레이션



작업 흐름 제어의 단일화: 랭체인은 이 독립적인 4대 모듈을 단 한 줄의 체인 표현식(`LCEL`)으로 간결하게 결합시켜, 중간 단계의 유실 없이 신뢰도 높은 데이터를 흐르게 유도합니다.

02. 랭체인 핵심 기능 (1) : 모델 추상화 & 기억 장치

기종 교체가 자유로운 유연한 설계 인프라와 단기/장기 연속성 보장을 위한 대화 관리 기법

01. MODEL ABSTRACTION



모델 추상화 (Model Abstraction)

LLM 공급사(OpenAI, Anthropic, Google 등)에 관계없이 단 하나의 공통 인터페이스로 코드 변경 없이 모델을 자유롭게 스왑(Swap)합니다.



직관적인 추상화 비유: 자동차 운전

"운전자는 엔진의 고유 연소 방식이나 기어박스의 내부 작동 구동 방식을 알지 못해도, 시동을 걸고 핸들을 돌려 전진(D)할 수 있습니다. 랭체인은 복잡한 AI 모델 내부를 자동차 새시처럼 표준 인터페이스로 감싸줍니다."

비즈니스 안정성 수호: 특정 AI 업체의 종속(Vendor Lock-in)을 막고 가격 인하 및 성능 고도화 시그널에 맞춰 즉각적인 백업 전환 환경을 확보합니다.

02. MEMORY INJECTION



기억 장치 추가 (Memory)

LLM API는 기본적으로 '무상태(Stateless)'로서 직전 대화를 완벽히 지워버립니다. 랭체인은 대화 이력을 버퍼에 누적해 AI가 문맥을 유지하게 조율합니다.



ChatGPT는 순수한 LLM이 아닙니다

"ChatGPT는 LLM 엔진 자체가 혼자 기억하는 것이 아닙니다. LLM이라는 핵심 두뇌 뒤에 대화 기록 장치(Memory System)를 연동한 별도의 완결된 애플리케이션인 것입니다."

대화 세션 흐름 관리: 토큰 소모를 방지하기 위해 이전 대화에서 중요한 핵심 요약본만 추출하여 압축 주입하는 지능형 메모리 버퍼를 지원합니다.

03. 랭체인 핵심 기능 (2) : RAG & 자율 에이전트

사내 데이터 보안 자산의 정밀 연계와 도구 자율 호출을 통한 고차원 해결 능력

03. RETRIEVAL-AUGMENTED GENERATION

외부 데이터 정밀 연계 (RAG)

LLM이 전혀 학습하지 못한 실시간 웹 정보, 최신 트렌드, 자사 내부 보안 규정 문서(PDF, DB, Notion)를 검색하여 답변에 반영시킵니다.

RAG 3-STEP PIPELINE

- 1 검색 (Retrieve): 사용자의 질의와 부합하는 관련 파일 문맥 검색
- 2 보강 (Augment): 검색된 실제 사실 데이터 프롬프트 자동 조립
- 3 생성 (Generate): 팩트 가이드를 기준으로 거짓(환각) 없는 답변 생성

할루시네이션 극복: 대규모 사내 문서 데이터베이스를 조각내어 최적의 구간을 가져오는 정합성 로더를 내장 지원합니다.

04. INTELLIGENT ACTION AGENT

도구 자율 결합 (Agent)

LLM이 스스로 판단하여 구글 검색, 이메일 전송, 슬랙 알림, 사내 Git 레포지토리 API 호출 등 외부 자원을 직접 작동하도록 돕습니다.

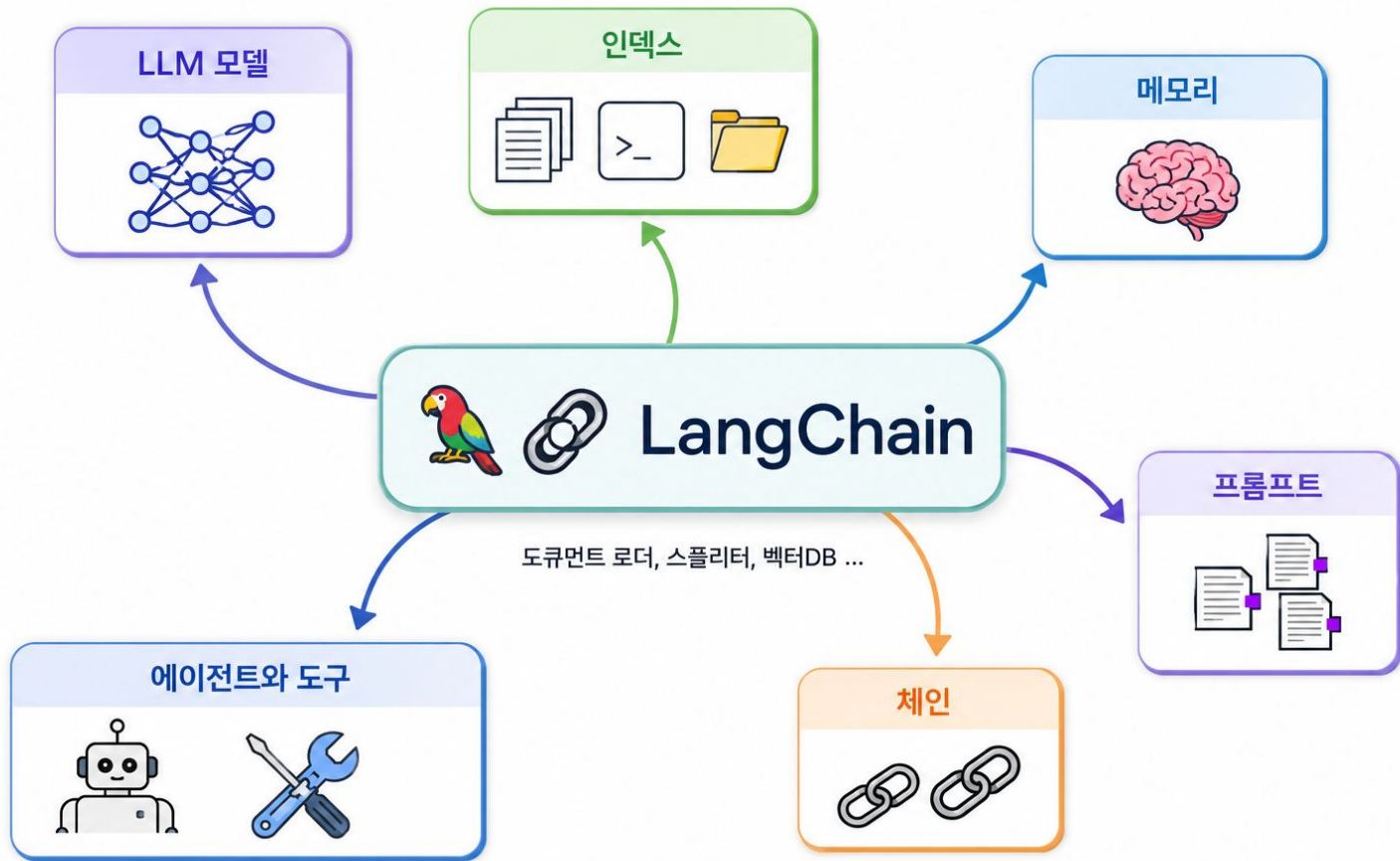
AGENT REASONING LOOP

THOUGHT: 사용자의 복합적인 전체 질문을 인식하고 판단 단계 설정

ACTION: Google Calendar API 호출 결정

OBS: 도구로부터 리턴받은 실행데이터를 확인하고 메일 자동 전송

자율 비서의 핵심 아키텍처: 스스로 도구 리스트를 탐색하고 다음 행동 양식을 유지하는 강한 계획형 추론 체인을 구성합니다.



01. RAG (검색 증강 생성) : 외부 지식과 지능의 결합

거대 언어 모델(LLM)이 가진 정보의 한계와 환각 현상(Hallucination)을 사내 정보 보정 검색망을 통해 물리적으로 격파하는 아키텍처입니다.

BACKGROUND PHILOSOPHY

시험장에 사내 규정 족보를 가지고 입장하는 시험자

순수 LLM은 아무런 자료 없이 자기 기억만으로 문제를 푸는 '폐쇄형 시험(Closed Book)' 상태입니다. 아무리 똑똑해도 모르는 사내 복지나 신규 스펙이 나오면 그럴듯한 거짓말을 지어내게 됩니다.

RAG는 질문과 일치하는 문서 조각들을 검색망을 통해 빠르게 수집한 뒤, 그 자료들을 LLM에게 건네며 '이 자료 안에서만 대답해(Open Book)'라고 지시하여 정확성을 보장합니다.

CORE ENGINE

RAG의 핵심은 자료 뭉치에서 가장 정답에 가까운 내용을 컴퓨터가 알아듣는 숫자의 크기로 변환하여 비교 탐색하는 '문장 임베딩(Sentence Embedding)' 기술입니다.

INTERACTIVE PIPELINE BLUEPRINT

RAG 아키텍처 수집-검색-생성 통합 데이터 파이프라인

외부 원천 문서를 조각내어 저장하고, 질문과 일치하는 벡터를 수집해 답변을 빚어내는 전 과정



파이프라인 통제: RAG 아키텍처는 원천 문서를 숫자 벡터 공간에 매핑해 두는 "사전 수집 과정(오프라인)"과, 질문이 들어왔을 때 관련 정보를 취합하여 LLM에게 주입하는 "실시간 추론 과정(온라인)"의 상호 작용으로 작동됩니다.

02. 데이터 구축 단계 : ① 청크 분할 및 ② 벡터 저장

사내 데이터 원천 자산을 기계가 고도로 인지할 수 있도록 가공하여 지식 정보 인프라를 구축하는 사전 작업

STEP 01. CHUNKING PIPELINE



① 데이터 준비 및 청크(Chunk) 분할

책이나 사내 법률/복지 문서처럼 긴 통 텍스트는 한 번에 모델에 집어넣기 비효율적이며 임베딩 시 중요한 맥락이 희석됩니다. 따라서 문맥과 가독성이 끊기지 않도록 글자 수(예: 300자~500자) 기준으로 텍스트를 정밀 컷팅합니다.

</> Chunking Simulation

Chunk 1 "...당사는 임직원들의 인락한 주거 복지를 제공하기 위하여 [주택 임차 자금 대출 지원 제도]를 상시 운용합니다..."

Chunk 2 "...임차 대출 한도는 최대 1억 원 이내이며 연 금리 1.5% 우대 금리를 대입해 소속 주임급 이상 사원에게 수혜 자격을 부여..."

슬라이딩 윈도우 기법: 문맥이 절단선에서 임의 유실되는 것을 막기 위해 청크 간 일정 영역(예: 50자~100자)을 겹치게 (Overlap) 분할하는 정확성을 확보합니다.

STEP 02. VECTOR DB STORAGE



② 문서 임베딩 및 벡터 DB 저장

쪼갠 각각의 청크 텍스트를 인공지능 임베딩 모델(예: OpenAI Ada, bge-m3)에 밀어 넣어, 고차원 공간 속의 실수 좌표 배열인 **숫자 벡터(Vector)**로 변환한 후 벡터 인덱스 DB에 영구 적재합니다.



주요 상용 벡터 데이터베이스 예시

Pinecone

Cloud Only

Milvus

Enterprise

Chroma DB

Lightweight

FAISS

Meta Open

고속 다차원 검색: 벡터 DB는 수억 개의 다차원 좌표 사이에서 공간적 거리가 유사한 문장을 0.01초 이내에 검색하는 K-최근접 이웃(KNN) 전용 색인 알고리즘을 수행합니다.

03. 실시간 추론 단계 : ③ 유사도 검색 및 ④ LLM 생성

사용자의 질문이 인입되는 실시간 상황에서, 질문 벡터와 가장 일치하는 문맥을 찾아내고 이를 조합해 무결 답변을 창조하는 과정

STEP 03. COSINE SIMILARITY MATCHING

③ 질문 임베딩 및 코사인 유사도 검색

사용자의 질문이 서버에 입력되면 즉시 동일한 임베딩 모델로 전향 변환합니다. 그 후 벡터 데이터베이스 속에서 질문 벡터와 **방향(각도)의 정합성이 가장 일치하는 코사인 유사도(Cosine Similarity)** 최상위 청크를 선별해 냅니다.

코사인 유사도 산출 공식

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{||\mathbf{A}|| \cdot ||\mathbf{B}||}$$

- **1.0 (0도):** 두 문장의 의미 완전 일치
- **0.0 (90도):** 독립적이고 무관한 문장
- **-1.0 (180도):** 정반대 및 상치되는 의미

콘텐츠 주입: 키워드가 100% 일치하지 않고 질문에 동의어 및 축약어가 섞여 있어도, "의미론적 유사도"를 정확히 감지해 관련 청크(Top 3~5)를 추출해 냅니다.

STEP 04. CONTEXT INJECTION & LLM GENERATION

④ 프롬프트 조립 및 LLM 생성 (답변 도출)

벡터 DB에서 찾아낸 실제 텍스트 조각들을 ****시스템 컨텍스트(배경 지식)****로 선언하고, 사용자의 본질적인 질문과 유기 결합하여 하나의 완성도 높은 프롬프트 패키지를 실시간 제작해 LLM에 정밀 투입합니다.

ASSEMBLED PROMPT FOR LLM

"당신은 사내 지식 도우미입니다.

****[컨텍스트: 복지 규정 2단계에서 검색된 실제 텍스트 청크]****

위의 신뢰할 수 있는 사실 컨텍스트 자료에만 의거하여 사용자의 질문인 *****우리 회사 복지 혜택이 뭐야?*****에 대해 왜곡이나 거짓(환각) 없이 정확히 한국어로 답변하십시오."

할루시네이션 종결: LLM은 학습된 두뇌 능력을 사용하되 지식 정보 소스는 오직 주어진 자료에만 복종하므로, 가짜 정보를 지어내는 인공지능의 취약점을 혁신적으로 종결합니다.

01. 임베딩(Embedding)이란? 의미의 정량적 수치화

텍스트, 이미지, 음성 등 인간의 데이터를 컴퓨터가 연산할 수 있도록 고정된 길이의 실수 벡터(Vector) 공간으로 좌표화하는 변환 기술입니다.

BASIC PRINCIPLE

"인간의 단어와 문맥을 수학적 주소로 치환하다"

컴퓨터는 "강아지"와 "개"라는 문자 자체만 보면 완전히 다른 글자로 취급합니다. 임베딩은 이들이 가진 **의미의 유사성**을 감지해 고차원 좌표계 상에서 매우 가까운 거리에 이웃하도록 정교하게 매핑합니다.

시맨틱 매칭의 핵심 비밀

사용자가 "고양이를 키우는 방법"을 검색했을 때, 문서 내에 해당 문장이 없고 "반려묘를 돌보는 법"만 적혀 있더라도, 두 문맥의 임베딩 벡터 거리가 매우 가깝기 때문에 AI는 이를 영리하게 찾아내어 매칭합니다.

VECTOR CONVERSION COMPARISON

자연어 문장의 다차원 벡터 변환 및 거리 대조 예시

서로 다른 세 문장이 임베딩 모델을 통과해 좌표값으로 사상되는 구조 분석

● "나는 강아지를 좋아한다."

반려동물 선호 그룹 (A)

[0.12, -0.53, 1.21, ..., 0.87]

● "나는 개를 좋아한다."

반려동물 선호 그룹 (A' - 유사도 최고)

[0.15, -0.50, 1.18, ..., 0.84]

● "오늘 주식 시장이 올랐다."

경제/금융 그룹 (B - 의미적 거리 매우 멀)

[-1.30, 0.90, -0.42, ..., -0.11]

분석 결과: 문장 1과 문장 2의 벡터 좌표값들은 소수점 자리가 극히 미세하게 차이 나며 거의 일치하는 방향 벡터를 이룹니다. 반면, 금융 도메인인 문장 3은 정반대의 마이너스 및 극단적 수치 좌표를 할당받아 공간적으로 완전히 이격됩니다.

02. 의미적 공간 배치 : 2차원 가상 벡터 공간 탐색기

아래 2D 좌표 평면은 다차원의 임베딩 공간을 알기 쉽게 2차원 축(\$X\$, \$Y\$)으로 축소한 모형입니다. 각 테마를 클릭하거나 검색하여 단어들의 군집화 구조를 체험하세요.

🔍 시맨틱 검색 테스트

유사한 개념어를 입력하거나 빠른 선택 키워드를 누르면, 해당 군집들이 실시간으로 밝게 빛나며 하이라이트됩니다.

검색

반려동물 군집

경제 군집

모빌리티 군집

📌 SELECTED VECTOR DATA

🚗 자동차

가상 차원 X (차원 1) 0.7800

가상 차원 Y (차원 2) -0.6200

군집 분류 (Semantic Label) 모빌리티 군집

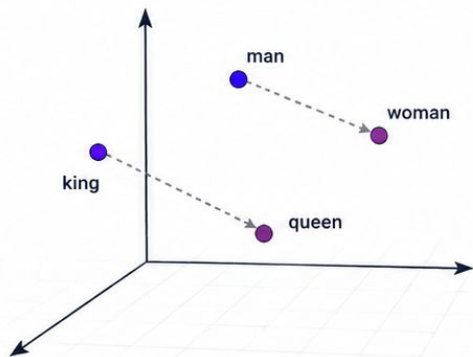
실제 임베딩 공간은 2차원이 아닌 1536차원 이상의 초공간입니다. 이를 인간의 눈에 맞추어 축소 정렬해도 의미적 거리는 그대로 연계 유지됩니다.

Dimension Y (Semantic Vertical Axis)



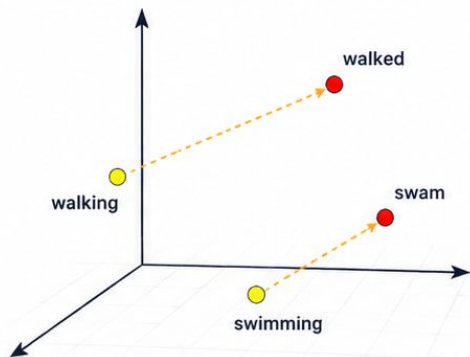
임베딩 벡터의 차원 예시

1. Male-Female



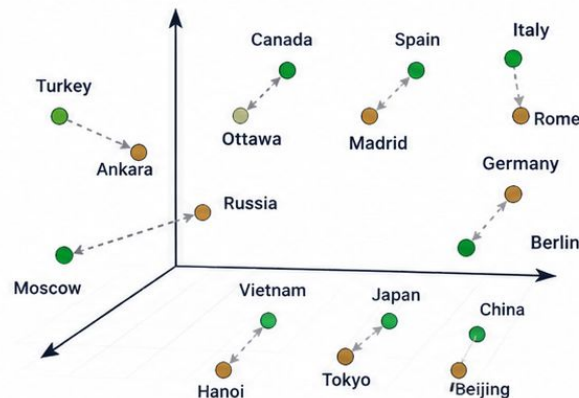
남성 ↔ 여성
의미적 관계가 벡터 공간에서
특정 방향으로 표현됨

2. Verb Tense



동사의 시제 변화
(현재형 ↔ 과거형, 현재분사 ↔ 과거분사)
역시 일정한 방향으로 표현됨

3. Country-Capital

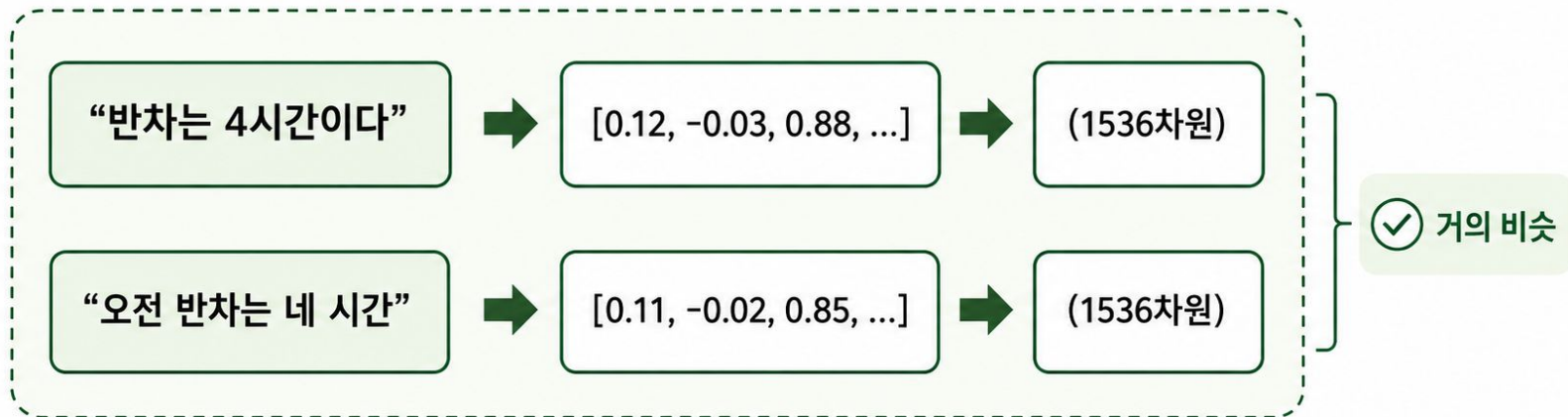


국가 ↔ 수도 관계
다양한 관계가 고차원 공간에서
일관된 패턴으로 표현됨



임베딩 벡터는 단어의 의미와 관계를 다차원 공간의 점으로 표현하여,
유사한 의미는 가까이, 특정 관계는 일정한 방향으로 포착할 수 있게 해줍니다.

— 문장 임베딩 예시 —



의미가 비슷한 문장은 임베딩 벡터도 비슷하고, 의미가 다른 문장은 임베딩 벡터도 다릅니다.

📦 임베딩 벡터의 '차원(Dimension)' 이해 : 의미를 숫자로 쪼개는 비유적 해석법

인간이 이해하는 사물의 속성들을 다차원의 숫자로 대칭 변환해보며 컴퓨터가 의미론적 유사 거리를 측정하는 근본 역학을 배웁니다.

CONCEPTUAL ANALOGY

"컴퓨터는 단어가 가진
'속성 점수표'를 채워가며
의미를 비교합니다."

실제 AI 모델은 수천 개 차원의 벡터 공간을 자동으로 형성하여 사물을 대수적으로 구획하고 그 값의 의미를 알수 없지만, 각 차원을 "특성 항목"으로 일대일 매핑해 이해해도 무방합니다.

❗ 주의: 실제 모델과의 차이점

실제 임베딩 모델에서는 각 차원(숫자)이 사람이 해석할 수 있는 명확한 의미("크기", "단맛")를 직접 지니지 않고, 학습 과정에서 형성된 고도화된 추상 특징을 나타낸다는 점을 이해하면 됩니다.

🏠 부동산 예시

Real Estate

1. 크기 / 2. 가격 / 3. 역과의 거리 / 4. 가구 수

🏠 아파트 [9, 8, 3, 10]

🏡 빌라 [5, 4, 3, 4]

> 두 단어 모두 주거용 부동산이므로 역과의 거리(3번째) 값은 일치합니다. 단, 아파트의 규모 및 가구 수가 훨씬 크게 사상됩니다.

🚗 자동차 예시

Mobility

1. 크기 / 2. 가격 / 3. 연비 / 4. 적재공간

🚗 세단 [6, 6, 8, 5]

🚙 SUV [9, 8, 5, 9]

> 둘 다 내연 모빌리티에 전반적으로 높은 유사도가 감지되지만, SUV는 세단 대비 크기와 적재용량에 가중치가 실립니다.

🍏 과일 예시

Agriculture

1. 단맛 / 2. 신맛 / 3. 수분 / 4. 크기

🍏 사과 [7, 5, 8, 6]

🍌 배 [8, 3, 9, 8]

> 두 객체 모두 수분이 풍부하고 달콤한 장미과 과일입니다. 다만, 배가 사과보다 조금 더 크고 신맛이 확연히 덜함을 알 수 있습니다.

03. 임베딩의 활용처 : RAG 검색 엔진 및 산업 적용

구축된 단어와 문장들의 고속 유사도 검색 성능을 이용해 현대적인 AI 시스템의 근간 파이프라인을 구축하는 사례들입니다.

RAG PIPELINE ROLE

RAG(검색 증강 생성) 시스템 속의 핵심 엔진

회사 문서가 10만 개 이상 존재하는 엔터프라이즈 환경에서의 탐색 방법

- 1 사내 10만 개 원천 문서 전량을 임베딩 모델을 통해 벡터로 전면 변환
- 2 인덱스화된 벡터 값들을 차세대 벡터 데이터베이스(Vector DB)에 일체 보관
- 3 사용자가 질문을 작성하면 그 질문 역시 실시간 임베딩으로 인스턴스 변환
- 4 질문 벡터와 공간 거리가 가장 인접한 정답 데이터 조각 검색
- 5 발견된 근사 텍스트 조각을 LLM 컨텍스트로 함께 묶어 환각 없이 출력 생성

속도의 혁신: 임베딩으로 변환하여 벡터 색인을 완료하면, 일반 텍스트 문장 10만 장을 실시간 정독하지 않고도 단 0.005초 만에 정확히 연관성이 가장 높은 문서만 골라냅니다.

INDUSTRY APPLICATIONS

임베딩 기술이 적용되는 주요 인공지능 산업

🔍 시맨틱 검색 (Search)

키워드 타자 매칭이 아닌 단어가 품고 있는 심도 깊은 의미 기준 고품질 매칭

🗣️ AI 에이전트 & 챗봇

고객 질문의 취지와 의도를 감지하여 최적의 정보 조각을 추출 보장

👍 고성능 추천 시스템

사용자의 시각 행동 및 구매 이력 경로를 좌표화해 유사 상품 표적 제안

📊 클러스터링 & 분류

정규 데이터가 매핑될 때 자동으로 유사 백락끼리 묶어 자동 색인

임베딩 핵심 정리: 임베딩은 컴퓨터가 한글이나 그림 같은 원시 소스를 인지하게 만드는 첫 관문이자 의미론적 수학의 절대 기준입니다.

임베딩과 RAG 검색의 본질

RAG의 핵심은 “질문 벡터와 가장 각도가 작은 문서 청크 상위 K개를 찾는 것”입니다.

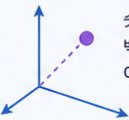
1. 임베딩이란?



임베딩은 문장을 숫자 목록(벡터)으로 바꾸는 과정입니다.

→ [0.21, -0.53, 0.12, ...]

2. 중요한 점



숫자 자체를 해석하는 것이 아니라, 벡터 사이의 거리와 방향(각도)을 이용합니다.



3. RAG 검색 로직



질문 벡터와 가장 각도가 작은 문서 청크 상위 K개를 찾아 응답 생성에 활용합니다.

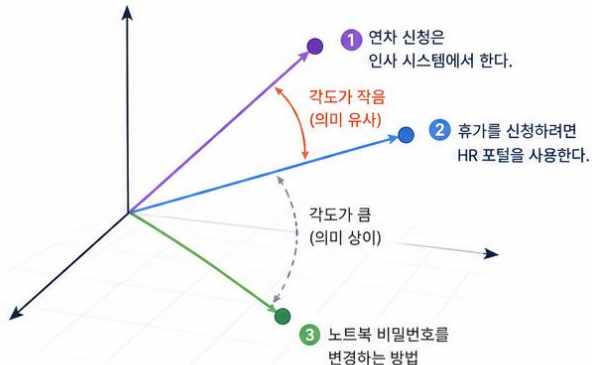
예시: 세 문장의 임베딩 공간

세 문장 예시

① 연차 신청은 인사 시스템에서 한다.

② “휴가를 신청하려면 HR 포털을 사용한다.”

③ “노트북 비밀번호를 변경하는 방법”



핵심 포인트

- ✓ 같은 임베딩 모델로 만든 벡터끼리는 두 벡터가 이루는 각도가 작을수록 의미가 비슷합니다.
- ✓ 첫 번째 문장과 두 번째 문장은 표현은 다르지만 의미가 비슷하여 임베딩 공간에서 가까운 위치에 놓입니다.
- ✓ 세 번째 문장은 주제가 다르므로 더 멀리 떨어진 위치에 놓입니다.



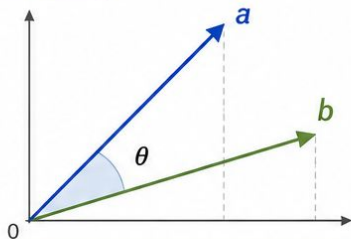
정리

임베딩은 문장을 벡터로 변환하고, RAG는 벡터 간의 거리와 방향(각도)을 기반으로 가장 관련성 높은 문서 청크를 찾아냅니다.

코사인 유사도 (Cosine Similarity)

두 벡터가 얼마나 같은 방향을 가리키는지 측정하는 지표 (크기의 영향 없이 **방향만 비교**)

1. 개념



θ : 두 벡터 사이의 각도
 $0^\circ \leq \theta \leq 180^\circ$

2. 수식

$$\cos(\theta) = \frac{a \cdot b}{|a| |b|}$$

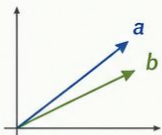
- $a \cdot b$: 벡터 a 와 b 의 내적 (dot product)
- $|a|, |b|$: 벡터 a, b 의 크기 (magnitude)
- $\cos(\theta)$: 코사인 유사도 ($-1 \leq \cos(\theta) \leq 1$)

3. 해석

1에 가까울수록	같은 방향 ($\theta \rightarrow 0^\circ$)
0에 가까울수록	직교(수직) 방향 ($\theta \rightarrow 90^\circ$)
-1에 가까울수록	반대 방향 ($\theta \rightarrow 180^\circ$)
0	유사도 없음 (방향 무관)

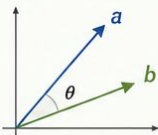
4. 예시

같은 방향
($\theta = 0^\circ$)



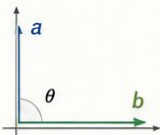
$$\cos(\theta) = 1$$

비슷한 방향
($0^\circ < \theta < 90^\circ$)



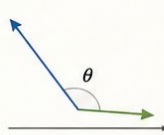
$$0 < \cos(\theta) < 1$$

직교 방향
($\theta = 90^\circ$)



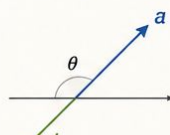
$$\cos(\theta) = 0$$

반대 방향에 가까움
($90^\circ < \theta < 180^\circ$)



$$-1 < \cos(\theta) < 0$$

반대 방향
($\theta = 180^\circ$)



$$\cos(\theta) = -1$$

5. 활용 예시

문서 유사도	두 문서의 단어 벡터가 얼마나 유사한지 측정
추천 시스템	사용자/아이템 벡터의 유사도 계산
정보 검색	검색 쿼리와 문서 벡터의 유사도로 관련 문서 랭킹
군집화	비슷한 방향의 벡터끼리 그룹화

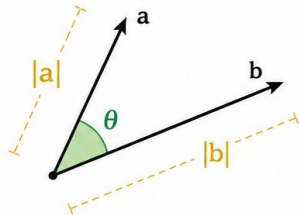


코사인 유사도는 벡터의 **크기(길이)**에는 영향을 받지 않고, 오직 **방향의 유사성**만을 측정합니다.

Dot Product of Vectors

Measures how much two vectors point in the same direction

1. Geometric Definition

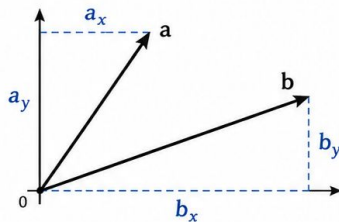


Geometric Formula

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos(\theta)$$

- $|\mathbf{a}|, |\mathbf{b}|$: magnitudes of vectors $\mathbf{a}$ and $\mathbf{b}$
- θ : angle between $\mathbf{a}$ and $\mathbf{b}$ ($0^\circ \leq \theta \leq 180^\circ$)

2. Component Form



Component Formula

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y$$

- a_x, a_y : components of vector $\mathbf{a}$
- b_x, b_y : components of vector $\mathbf{b}$

3. Key Relationship

$$|\mathbf{a}| |\mathbf{b}| \cos(\theta) = a_x b_x + a_y b_y$$

4. Cosine Formula

$$\cos(\theta) = \frac{a_x b_x + a_y b_y}{|\mathbf{a}| |\mathbf{b}|}$$

★ Useful for finding the angle between vectors.